Лабораторная работа № 4

по дисциплине
«Технологии промышленной разработки программного обеспечения.
Основные понятия»

Подсказка в экранной таблице

Выполнил: студент группы М26-504

Гришанов Николай Вадимович

Москва — 2026

Содержание

1 Описание работы **3**

1.1 Цель и постановка 3

1.2 Недостатки базового образца 3

1.3 Принятое решение 4

2 Код **5**

2.1 Точка входа 5

2.2 Форма, список и подсказка 5

2.3 Разметка окна 15

2.4 Съёмка сценариев 17

3 Описание написанного кода **20**

3.1 Form1 20

3.2 Names и Prompt 20

3.3 DemoShots 21

4 Описание тестовых сценариев **21**

5 Тестовые сценарии **22**

5.1 Старт 22

5.2 Команды меню 23

5.3 Подсказка под ячейкой «Название» 24

5.4 Продвижение селекции при вводе 25

5.5 Стрелка вниз 26

5.6 Enter переносит строку подсказки 27

5.7 Новая строка попадает в подсказку 28

5.8 Отказ по Escape 29

5.9 Пустой Enter оставляет подсказку 30

5.10 Щелчок по строке подсказки 31

5.11 Подсказка столбца «Тип» 32

5.12 Расширение комбинированного столбца 33

5.13 Отдельный список столбца NN 34

6 Документация **35**

6.1 Руководство оператора 35

6.2 Клавиши 36

6.3 Сборка 36

6.4 Ограничения 37

6.5 Состав поставки 37

Заключение **37**

1. Описание работы

1.1. Цель и постановка

Цель работы — разработать типовое решение подсказки для экранной таблицы Windows Forms. Подсказка отличается от штатного выпадающего списка тем, что её состав расширяется во время работы: строка, введённая в ячейку и отсутствующая в списке, становится новой строкой подсказки. По методическим указаниям на форме стоят две таблицы DataGridView. Рабочая таблица dataGridView2 содержит столбцы NN, «Название» и «Тип». Столбец «Тип» — штатный ComboBoxColumn со значениями «Зверь», «Птица», «Рыба», «Гад». Вторая таблица, dgwNamesPrompt, скрыта до редактирования и служит окном подсказки: без заголовков столбцов и строк. Список подсказки — List<Names> с одним свойством Name, привязанный через BindingSource. Меню содержит команды «Создать список», «Подключить список» и «Упорядочить». Универсальная подсказка покрывает такие прецеденты:

1. после первой литеры или повторного щелчка по уже выделенной ячейке подсказка открывается, ячейка остаётся в режиме редактирования;
2. литеры попадают в ячейку, а выделение в подсказке сдвигается по возрастанию к строке, которая не меньше введённого текста;
3. стрелки вверх и вниз двигают выделение, Enter переносит выбранную строку в ячейку, Escape отказывается от подсказки;
4. Enter по набранному тексту, которого в списке нет, копирует его в подсказку, сортирует список и оставляет подсказку открытой;
5. щелчок левой кнопкой по строке подсказки копирует её в ячейку;
6. подсказка работает для любого столбца рабочей таблицы, а не только для столбца «Название».

1.2. Недостатки базового образца

Текст методички задаёт класс Prompt и обработчики формы, но в приложенном коде сценарии расходятся с реализацией.

- Подсказка открывается только при ColumnIndex == 1. Остальные столбцы остаются без неё, хотя пункт 3.7 требует подсказку к любому столбцу.
- OpenPrompt обращается к Rows[0] сразу после открытия. Пустой список да-

ёт исключение, хотя сценарии 1–2 как раз начинаются с пустой подсказки.

- Обработчики мыши и клавиатуры подписываются при каждом открытии. ClosePrompt снимает их только когда Active ложно, поэтому при наведении указателя подсказка не закрывается и подписки копятся.
- В Spec_PreKeyDown перенос строки из подсказки в ячейку завязан на Alt+Enter. В постановке выбор выполняется стрелками и обычным Enter.
- Продвижение селекции в Spec_KeyPress идёт только вперёд по CurPromptrow. Стирание символа не возвращает выделение к более ранней строке.
- «Упорядочить» и StoreNewCellValue подменяют источник привязки новым списком и не обновляют поле vns. Следующее пополнение и команда меню работают уже с другим объектом.
- «Подключить список» вызывает Rows.Clear() у таблицы, которая может быть уже привязана к данным. Для привязанного DataGridView это исключение.
- Штатный ComboBoxColumn не принимает значение, которого нет в Items. Новая строка подсказки в столбце «Тип» без дописывания в список столбца завершается ошибкой данных.

1.3. Принятое решение

Окно и класс подсказки оставлены в одном модуле Form1.cs, как в приложении к методичке: пространство имён TipTop, таблица dataGridView2, окно dgwNamesPrompt, статус toolStripStatusLabel2. Исправлено то, что мешало прецедентам, и подсказка обобщена на все столбцы. У каждого столбца свой List<Names> и свой BindingSource. Для столбца «Название» (индекс 1) это общие Form1.vns и Form1.vbs, поэтому три команды меню по-прежнему управляют именно этим списком. Столбцы NN и «Тип» копят собственные строки. При первом открытии столбца «Тип» в его список копируются четыре значения комбинированного столбца. Подсказка ставится под текущую ячейку и поднимается над таблицей. Если снизу не хватает места, она открывается над ячейкой. Пока строк не больше восьми, полоса прокрутки скрыта и видны все элементы. Пустой список показывает компактное пустое окно, а не обращение к несуществующей строке. Клавиатура обрабатывается на элементе редактирования. PreviewKeyDown помечает Enter, Escape и стрелки как клавиши ввода, чтобы таблица не уводила их на соседнюю ячейку. Стрелка двигает выделение подсказки. Следующий Enter переносит выделенную строку в ячейку и завершает

редактирование. Enter без предварительных стрелок заносит набранный текст в список столбца, сортирует его и оставляет ячейку в редактировании. Пустой Enter завершает редактирование и оставляет подсказку на экране. Escape возвращает прежнее значение ячейки и прячет подсказку. Щелчок по строке подсказки запоминается в MouseDown и применяется после текущего сообщения мыши: так значение успевает попасть в ячейку и не теряется из-за потери фокуса редактором. Пока указатель над подсказкой, флаг Active не даёт CellEndEdit закрыть окно раньше щелчка. Новая непустая строка добавляется, только если такой ещё нет. Список заново сортируется культурой текущей локали, той же, которой пользуется OrderBy. Для комбинированного столбца то же значение добавляется в Items, поэтому ячейка «Тип» принимает введённое слово. Поиск селекции при наборе каждый раз начинается с первой строки, поэтому Backspace сдвигает выделение назад.

2. Код

Решение — один проект Windows Forms SDK-стиля на .NET 10, пространство имён TipTop. Файл решения: TipTop/TipTop.sln (рядом лежит TipTop.slnx).

2.1. Точка входа

```
using System;
using System.Windows.Forms;

namespace TipTop
{
  static class Program
  {
    [STAThread]
    static void Main(string[] args)
    {
      ApplicationConfiguration.Initialize();
      if (args.Length > 0 && args[0].StartsWith("--demo-out"))
      {
        string dir = args[0].Contains('=')
          ? args[0].Substring(args[0].IndexOf('=') + 1)
          : (args.Length > 1 ? args[1] : "images");
        Environment.ExitCode = DemoShots.Run(dir);
        return;
      }
      Application.Run(new Form1());
    }
  }
}
```

2.2. Форма, список и подсказка

```
// Лабораторная работа 4.
// Экранная таблица: ввод с клавиатуры, ComboBox-столбец и расширяемая подсказка.
// Подсказка ведётся отдельно для каждого столбца и пополняется новыми строками.
```

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;

namespace TipTop
{
 public partial class Form1 : Form
 {
  public Form1()
  {
   InitializeComponent();
   BuildTable();
   Stat = toolStripStatusLabel2;
   toolStripStatusLabel2.Text = "Создайте список, подключите его и редактируйте ячейки. Стрелки и Enter выбирают подсказку, Esc
   ↪   отменяет.";
   Resize += delegate { if (Prompt.Opened) Prompt.AdjustPrompt(); };
   dataGridView2.CurrentCellChanged += dataGridView2_CurrentCellChanged;
  }

  private void BuildTable()
  {
   dataGridView2.AutoGenerateColumns = false;
   dataGridView2.AllowUserToAddRows = true;
   dataGridView2.MultiSelect = false;
   dataGridView2.SelectionMode = DataGridViewSelectionMode.CellSelect;
   dataGridView2.EditMode = DataGridViewEditMode.EditOnKeystrokeOrF2;
   dataGridView2.Columns.Clear();

   dataGridView2.Columns.Add(new DataGridViewTextBoxColumn
   {
    Name = "colNN",
    HeaderText = "NN",
    Width = 80
   });
   dataGridView2.Columns.Add(new DataGridViewTextBoxColumn
   {
    Name = "colName",
    HeaderText = "Название",
    Width = 240
   });

   DataGridViewComboBoxColumn typeColumn = new DataGridViewComboBoxColumn
   {
    Name = "colType",
    HeaderText = "Тип",
    Width = 160,
    FlatStyle = FlatStyle.Flat,
    DisplayStyle = DataGridViewComboBoxDisplayStyle.ComboBox
   };
   typeColumn.Items.AddRange(new object[] { "Зверь", "Птица", "Рыба", "Гад" });
   dataGridView2.Columns.Add(typeColumn);

   for (int i = 0; i < 8; i++)
    dataGridView2.Rows.Add();
  }

  private void создатьСписокToolStripMenuItem_Click(object sender, EventArgs e)
  {
   AddUnique(vns, "Argon");
   AddUnique(vns, "Xenon");
   AddUnique(vns, "Cripton");
   if (ReferenceEquals(vbs.DataSource, vns))
    vbs.ResetBindings(false);
   Prompt.NotifyListChanged(1);
   toolStripStatusLabel2.Text = "Список создан: Argon, Xenon, Cripton";
  }

  private void подключитьСписокToolStripMenuItem_Click(object sender, EventArgs e)
  {
   if (dgwNamesPrompt.DataSource == null)
    dgwNamesPrompt.Rows.Clear();
   dgwNamesPrompt.DataSource = vbs;
   vbs.DataSource = vns;
   Prompt.NotifyListChanged(1);
   toolStripStatusLabel2.Text = "Список подключён (" + vns.Count + ")";
  }

  private void упорядочитьToolStripMenuItem_Click(object sender, EventArgs e)
  {
   IEnumerable<Names> query = vns.OrderBy(nmes => nmes.Name);
   List<Names> vons = new List<Names>();
   foreach (Names nm in query)
    vons.Add(new Names { Name = nm.Name });
   vns = vons;
```

```
  vbs.DataSource = vons;
  Prompt.NotifyListChanged(1);
  toolStripStatusLabel2.Text = "Список упорядочен";
}

public static List<Names> vns = new List<Names>();
public static BindingSource vbs = new BindingSource();
public static ToolStripStatusLabel Stat;
static Dictionary<int, List<Names>> columnLists = new Dictionary<int, List<Names>>();
static Dictionary<int, BindingSource> columnSources = new Dictionary<int, BindingSource>();

public static List<Names> GetList(int col)
{
  if (col == 1)
    return vns;
  List<Names> list;
  if (!columnLists.TryGetValue(col, out list))
  {
    list = new List<Names>();
    columnLists[col] = list;
  }
  return list;
}

public static void SetList(int col, List<Names> list)
{
  if (col == 1)
    vns = list;
  else
    columnLists[col] = list;
  GetSource(col).DataSource = list;
}

public static BindingSource GetSource(int col)
{
  if (col == 1)
    return vbs;
  BindingSource bs;
  if (!columnSources.TryGetValue(col, out bs))
  {
    bs = new BindingSource();
    columnSources[col] = bs;
  }
  return bs;
}

static void AddUnique(List<Names> list, string name)
{
  foreach (Names nm in list)
  {
    if (string.Equals((nm.Name ?? "").Trim(), name, StringComparison.CurrentCulture))
      return;
  }
  list.Add(new Names { Name = name });
}

private void dataGridView2_CellBeginEdit(object sender, DataGridViewCellCancelEventArgs e)
{
  if (e.RowIndex < 0 || e.ColumnIndex < 0)
    return;
  Prompt.OpenPrompt(toolStripStatusLabel2, dgwNamesPrompt, dataGridView2, e);
}

private void dataGridView2_CellEndEdit(object sender, DataGridViewCellEventArgs e)
{
  Prompt.DetachEditControl();
  if (Prompt.Active && !Prompt.FromPrompt)
    return;
  if (!Prompt.Cancelled && !Prompt.FromPrompt)
    Prompt.StoreNewCellValue(e, dataGridView2, dgwNamesPrompt);
  if (!Prompt.KeepOpen)
    Prompt.ForceClose();
  Prompt.Cancelled = false;
  Prompt.FromPrompt = false;
  Prompt.KeepOpen = false;
  Prompt.Active = false;
}

private void dataGridView2_EditingControlShowing(object sender, DataGridViewEditingControlShowingEventArgs e)
{
  Prompt.Attach(e.Control);
  if (Prompt.CaretAtStart || Control.MouseButtons == MouseButtons.Left)
    Prompt.PlaceCaretAtStart();
  Prompt.CaretAtStart = false;
  if (e.Control is ComboBox combo)
  {
    combo.DropDownStyle = ComboBoxStyle.DropDown;
```

```
      combo.AutoCompleteMode = AutoCompleteMode.None;
      combo.AutoCompleteSource = AutoCompleteSource.None;
      combo.DroppedDown = false;
      BeginInvoke(new Action(delegate
      {
        if (!combo.IsDisposed)
          combo.DroppedDown = false;
      }));
    }
    toolStripStatusLabel2.Text = "table ED is initiated";
  }

  private void dataGridView2_CellMouseDown(object sender, DataGridViewCellMouseEventArgs e)
  {
    if (e.Button != MouseButtons.Left || e.RowIndex < 0 || e.ColumnIndex < 0)
      return;
    bool wasCurrent = dataGridView2.CurrentCell != null
      && dataGridView2.CurrentCell.RowIndex == e.RowIndex
      && dataGridView2.CurrentCell.ColumnIndex == e.ColumnIndex;
    if (!wasCurrent || dataGridView2.IsCurrentCellInEditMode)
      return;
    Prompt.CaretAtStart = true;
    BeginInvoke(new Action(delegate
    {
      if (!dataGridView2.IsCurrentCellInEditMode
        && dataGridView2.CurrentCell != null
        && dataGridView2.CurrentCell.RowIndex == e.RowIndex
        && dataGridView2.CurrentCell.ColumnIndex == e.ColumnIndex)
        dataGridView2.BeginEdit(false);
    }));
  }

  private void dataGridView2_CellValidating(object sender, DataGridViewCellValidatingEventArgs e)
  {
    if (dataGridView2.Columns[e.ColumnIndex] is DataGridViewComboBoxColumn col)
      Prompt.EnsureComboItem(col, Convert.ToString(e.FormattedValue));
  }

  private void dataGridView2_DataError(object sender, DataGridViewDataErrorEventArgs e)
  {
    e.ThrowException = false;
  }

  private void dataGridView2_Scroll(object sender, ScrollEventArgs e)
  {
    if (Prompt.Opened)
      Prompt.AdjustPrompt();
  }

  private void dataGridView2_CurrentCellChanged(object sender, EventArgs e)
  {
    if (!Prompt.Opened || dataGridView2.IsCurrentCellInEditMode)
      return;
    DataGridViewCell cell = dataGridView2.CurrentCell;
    bool same = cell != null && cell.RowIndex == Prompt.curRow && cell.ColumnIndex == Prompt.curCell;
    if (same || (Prompt.Sticky && cell == null))
      return;
    Prompt.ForceClose();
  }
}

public class Names
{
  public string Name { get; set; }
}

class Prompt
{
  public static TextBox txt;
  public static Control edit;
  public static Rectangle vr;
  public static int curRow;
  public static int curCell;
  public static bool Active = false;
  public static bool Opened = false;
  public static bool KeepOpen = false;
  public static bool Cancelled = false;
  public static bool FromPrompt = false;
  public static bool CaretAtStart = false;
  public static bool Sticky = false;
  public static int CurPromptrow = 0;
  static bool mouseApplyQueued;
  static bool navigatedByArrows;
  static bool wired;
  static DataGridView vDGW;
  public static DataGridView vPrompt;
  public static ToolStripStatusLabel vStatus;
```

```
public static void OpenPrompt(ToolStripStatusLabel status, DataGridView dgwPrompt, DataGridView dgw, DataGridViewCellCancelEventArgs
↪   e)
{
 curRow = e.RowIndex;
 curCell = e.ColumnIndex;
 vDGW = dgw;
 vPrompt = dgwPrompt;
 vStatus = status;
 CurPromptrow = 0;
 navigatedByArrows = false;
 Cancelled = false;
 FromPrompt = false;
 Active = false;
 KeepOpen = false;
 Sticky = false;
 BindColumn(curCell);
 Wire();
 AdjustPrompt();
 if (vPrompt.Rows.Count > 0)
  vPrompt.ClearSelection();
 Opened = true;
 vPrompt.Visible = true;
 vPrompt.BringToFront();
 if (vStatus != null)
  vStatus.Text = "Подсказка открыта: " + dgw.Columns[curCell].HeaderText;
}

static void BindColumn(int col)
{
 List<Names> list = Form1.GetList(col);
 if (list.Count == 0 && vDGW.Columns[col] is DataGridViewComboBoxColumn combo)
 {
  foreach (object item in combo.Items)
  {
   string text = Convert.ToString(item);
   if (!string.IsNullOrWhiteSpace(text))
    list.Add(new Names { Name = text.Trim() });
  }
 }
 BindingSource bs = Form1.GetSource(col);
 bs.DataSource = list;
 vPrompt.DataSource = bs;
 if (vPrompt.Columns.Count > 0)
  vPrompt.Columns[0].AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill;
}

public static void AdjustPrompt()
{
 if (vDGW == null || vPrompt == null || vPrompt.Parent == null)
  return;
 Rectangle cell = vDGW.GetCellDisplayRectangle(curCell, curRow, true);
 if (cell.Width <= 0 || cell.Height <= 0)
 {
  vPrompt.Visible = false;
  return;
 }
 Point below = vPrompt.Parent.PointToClient(vDGW.PointToScreen(new Point(cell.Left, cell.Bottom)));
 int width = Math.Max(cell.Width, 120);
 int rowH = vPrompt.RowTemplate.Height > 0 ? vPrompt.RowTemplate.Height : 24;
 if (vPrompt.Rows.Count > 0 && vPrompt.Rows[0].Height > 0)
  rowH = vPrompt.Rows[0].Height;
 int visibleRows = vPrompt.Rows.Count == 0 ? 1 : Math.Min(vPrompt.Rows.Count, 8);
 int height = visibleRows * rowH + 8;
 vPrompt.ScrollBars = vPrompt.Rows.Count <= 8 ? ScrollBars.None : ScrollBars.Vertical;
 int minTop = 0;
 int maxBottom = vPrompt.Parent.ClientSize.Height;
 foreach (Control sibling in vPrompt.Parent.Controls)
 {
  if (sibling is MenuStrip menu && menu.Visible)
   minTop = Math.Max(minTop, menu.Bottom);
  if (sibling is StatusStrip status && status.Visible)
   maxBottom = Math.Min(maxBottom, status.Top);
 }
 int left = below.X;
 int top = below.Y;
 if (top + height > maxBottom)
 {
  int above = vPrompt.Parent.PointToClient(vDGW.PointToScreen(new Point(cell.Left, cell.Top))).Y - height;
  if (above >= minTop)
   top = above;
 }
 if (top < minTop)
  top = minTop;
 if (left + width > vPrompt.Parent.ClientSize.Width)
  left = Math.Max(0, vPrompt.Parent.ClientSize.Width - width);
 if (left < 0)
```

```
    left = 0;
  vPrompt.Bounds = new Rectangle(left, top, width, height);
  vr = vPrompt.Bounds;
  if (Opened)
    vPrompt.Visible = true;
}

public static void NotifyListChanged(int col)
{
  if (!Opened || curCell != col)
    return;
  AdjustPrompt();
}

public static void ClosePrompt()
{
  if (Active)
    return;
  ForceClose();
}

public static void ForceClose()
{
  Unwire();
  DetachEditControl();
  Opened = false;
  Active = false;
  KeepOpen = false;
  Sticky = false;
  mouseApplyQueued = false;
  navigatedByArrows = false;
  if (vPrompt != null)
    vPrompt.Visible = false;
}

static void Wire()
{
  if (wired || vPrompt == null)
    return;
  vPrompt.KeyDown += prompt_KeyDown;
  vPrompt.CellMouseClick += prompt_CellMouseClick;
  vPrompt.MouseDown += prompt_MouseDown;
  vPrompt.MouseEnter += prompt_MouseEnter;
  vPrompt.MouseLeave += prompt_MouseLeave;
  wired = true;
}

static void Unwire()
{
  if (!wired || vPrompt == null)
    return;
  vPrompt.KeyDown -= prompt_KeyDown;
  vPrompt.CellMouseClick -= prompt_CellMouseClick;
  vPrompt.MouseDown -= prompt_MouseDown;
  vPrompt.MouseEnter -= prompt_MouseEnter;
  vPrompt.MouseLeave -= prompt_MouseLeave;
  wired = false;
}

static void prompt_MouseEnter(object sender, EventArgs e)
{
  Active = true;
  if (vStatus != null)
    vStatus.Text = "Mouse enters prompt";
}

static void prompt_MouseLeave(object sender, EventArgs e)
{
  Active = false;
  if (FromPrompt || !Opened)
    return;
  if (vStatus != null)
    vStatus.Text = "Mouse leaves prompt";
  if (vDGW != null && !vDGW.IsCurrentCellInEditMode && curRow >= 0 && curRow < vDGW.Rows.Count)
  {
    string loc = Convert.ToString(vDGW.Rows[curRow].Cells[curCell].Value);
    StoreText(curCell, loc);
  }
}

static void prompt_MouseDown(object sender, MouseEventArgs e)
{
  if (e.Button != MouseButtons.Left || vPrompt == null || vDGW == null)
    return;
  DataGridView.HitTestInfo hit = vPrompt.HitTest(e.X, e.Y);
  if (hit.RowIndex < 0)
    return;
```

```
  string value = CellText(hit.RowIndex);
  if (value.Length == 0)
    return;
  mouseApplyQueued = true;
  FromPrompt = true;
  Active = true;
  if (edit != null)
    EditText = value;
  if (curCell >= 0 && curCell < vDGW.Columns.Count)
    EnsureComboItem(vDGW.Columns[curCell] as DataGridViewComboBoxColumn, value);
  if (vStatus != null)
    vStatus.Text = "write from prompt to table";
  vDGW.BeginInvoke(new Action(delegate
  {
    mouseApplyQueued = false;
    ApplyToCell(value);
  }));
}

static void prompt_CellMouseClick(object sender, DataGridViewCellMouseEventArgs e)
{
  if (mouseApplyQueued || e.RowIndex < 0 || !Opened)
    return;
  ApplyToCell(CellText(e.RowIndex));
}

static void prompt_KeyDown(object sender, KeyEventArgs e)
{
  if (e.KeyCode == Keys.Enter && vPrompt != null && vPrompt.CurrentRow != null)
  {
    ApplyToCell(Convert.ToString(vPrompt.CurrentRow.Cells[0].Value));
    e.Handled = true;
  }
  else if (e.KeyCode == Keys.Escape)
  {
    CancelEditAndHide();
    e.Handled = true;
  }
}

public static void Spec_PreKeyDown(object sender, PreviewKeyDownEventArgs e)
{
  if (e.KeyCode == Keys.Enter || e.KeyCode == Keys.Escape || e.KeyCode == Keys.Up || e.KeyCode == Keys.Down)
    e.IsInputKey = true;
}

public static void Spec_KeyDown(object sender, KeyEventArgs e)
{
  if (e.KeyCode == Keys.Down)
  {
    MovePrompt(1);
    e.Handled = true;
    e.SuppressKeyPress = true;
  }
  else if (e.KeyCode == Keys.Up)
  {
    MovePrompt(-1);
    e.Handled = true;
    e.SuppressKeyPress = true;
  }
  else if (e.KeyCode == Keys.Enter)
  {
    e.Handled = true;
    e.SuppressKeyPress = true;
    if (navigatedByArrows && vPrompt != null && vPrompt.Rows.Count > 0)
      ApplyToCell(CellText(CurPromptrow));
    else
      CommitTypedText();
  }
  else if (e.KeyCode == Keys.Escape)
  {
    e.Handled = true;
    e.SuppressKeyPress = true;
    CancelEditAndHide();
  }
}

public static void Spec_KeyPress(object sender, KeyPressEventArgs e)
{
  if (char.IsControl(e.KeyChar))
    return;
  if (SelectionLength > 0)
    EditText = "";
  string vcur = EditText + e.KeyChar;
  navigatedByArrows = false;
  Advance(vcur);
}
```

```
public static void Spec_KeyUp(object sender, KeyEventArgs e)
{
 if (e.KeyCode == Keys.Back || e.KeyCode == Keys.Delete)
 {
  navigatedByArrows = false;
  Advance(EditText.Trim());
 }
}

static void CommitTypedText()
{
 string text = (EditText ?? "").Trim();
 if (text.Length == 0)
 {
  Cancelled = true;
  KeepOpen = true;
  Sticky = true;
  Active = false;
  if (vStatus != null)
    vStatus.Text = "Ячейка закрыта, подсказка видима";
  if (vDGW != null && vDGW.IsCurrentCellInEditMode)
    vDGW.EndEdit();
  else
    Opened = true;
  return;
 }
 StoreText(curCell, text);
 navigatedByArrows = false;
 CurPromptrow = 0;
 KeepOpen = true;
 Opened = true;
 ClearPromptSelection();
 if (vPrompt != null)
 {
  vPrompt.Visible = true;
  vPrompt.BringToFront();
 }
 AdjustPrompt();
 if (vStatus != null)
  vStatus.Text = "Строка занесена в подсказку";
}

static void CancelEditAndHide()
{
 Cancelled = true;
 Active = false;
 KeepOpen = false;
 Sticky = false;
 if (vDGW != null && vDGW.IsCurrentCellInEditMode)
 {
  try
  {
   RestoreEditorToCell();
   vDGW.CancelEdit();
   if (vDGW.IsCurrentCellInEditMode)
     vDGW.EndEdit();
  }
  catch (InvalidOperationException)
  {
   vDGW.BeginInvoke(new Action(FinishCancel));
   return;
  }
 }
 ForceClose();
 if (vStatus != null)
  vStatus.Text = "Подсказка скрыта";
}

static void FinishCancel()
{
 try
 {
  if (vDGW != null && vDGW.IsCurrentCellInEditMode)
  {
   RestoreEditorToCell();
   vDGW.CancelEdit();
   vDGW.EndEdit();
  }
 }
 catch (InvalidOperationException)
 {
 }
 ForceClose();
 if (vStatus != null)
  vStatus.Text = "Подсказка скрыта";
}
```

```
static void RestoreEditorToCell()
{
 if (vDGW == null || curRow < 0 || curRow >= vDGW.Rows.Count)
   return;
 string original = Convert.ToString(vDGW.Rows[curRow].Cells[curCell].Value) ?? "";
 if (edit != null)
   EditText = original;
}

static void MovePrompt(int delta)
{
 if (vPrompt == null || vPrompt.Rows.Count == 0)
   return;
 HideComboDropDown();
 if (!navigatedByArrows && delta > 0 && !HasSelection())
   CurPromptrow = 0;
 else
   CurPromptrow += delta;
 if (CurPromptrow < 0)
   CurPromptrow = 0;
 if (CurPromptrow >= vPrompt.Rows.Count)
   CurPromptrow = vPrompt.Rows.Count - 1;
 navigatedByArrows = true;
 Active = true;
 vPrompt.ClearSelection();
 vPrompt.Rows[CurPromptrow].Selected = true;
 vPrompt.CurrentCell = vPrompt.Rows[CurPromptrow].Cells[0];
 if (vStatus != null)
   vStatus.Text = "Подсказка: " + CellText(CurPromptrow);
}

static bool HasSelection()
{
 return vPrompt != null && vPrompt.SelectedRows.Count > 0;
}

static void Advance(string vcur)
{
 if (vPrompt == null || vPrompt.Rows.Count == 0)
   return;
 vcur = vcur ?? "";
 vPrompt.ClearSelection();
 if (vcur.Trim().Length == 0)
 {
  CurPromptrow = 0;
  return;
 }
 CurPromptrow = 0;
 while (CurPromptrow < vPrompt.Rows.Count
  && string.Compare(vcur, CellText(CurPromptrow), StringComparison.CurrentCulture) > 0)
  CurPromptrow++;
 if (CurPromptrow < vPrompt.Rows.Count
  && string.Compare(vcur, CellText(CurPromptrow), StringComparison.CurrentCulture) <= 0)
 {
  vPrompt.Rows[CurPromptrow].Selected = true;
  vPrompt.CurrentCell = vPrompt.Rows[CurPromptrow].Cells[0];
  if (vStatus != null)
    vStatus.Text = "Подсказка: " + CellText(CurPromptrow);
 }
 else
  CurPromptrow = 0;
}

public static void StoreNewCellValue(DataGridViewCellEventArgs e, DataGridView dgw, DataGridView dgwPrompt)
{
 if (e.RowIndex < 0 || e.RowIndex >= dgw.Rows.Count)
   return;
 string locnm = Convert.ToString(dgw.Rows[e.RowIndex].Cells[e.ColumnIndex].Value);
 StoreText(e.ColumnIndex, locnm);
}

public static void StoreText(int column, string value)
{
 value = (value ?? "").Trim();
 if (value.Length == 0)
   return;
 List<Names> list = Form1.GetList(column);
 bool found = false;
 foreach (Names nm in list)
 {
  if ((nm.Name ?? "").Trim() == value)
  {
   found = true;
   break;
  }
 }
```

```
  if (!found)
    list.Add(new Names { Name = value });
  List<Names> sorted = list
    .OrderBy(nmes => nmes.Name ?? "", StringComparer.CurrentCulture)
    .Select(nm => new Names { Name = nm.Name })
    .ToList();
  Form1.SetList(column, sorted);
  if (vDGW != null && column >= 0 && column < vDGW.Columns.Count)
    EnsureComboItem(vDGW.Columns[column] as DataGridViewComboBoxColumn, value);
  if (Opened && curCell == column)
    AdjustPrompt();
}

public static void EnsureComboItem(DataGridViewComboBoxColumn col, string value)
{
  value = (value ?? "").Trim();
  if (col == null || value.Length == 0)
    return;
  foreach (object item in col.Items)
  {
    if (Convert.ToString(item) == value)
      return;
  }
  col.Items.Add(value);
}

static void ApplyToCell(string value)
{
  value = (value ?? "").Trim();
  if (value.Length == 0 || vDGW == null)
    return;
  if (curCell >= 0 && curCell < vDGW.Columns.Count)
    EnsureComboItem(vDGW.Columns[curCell] as DataGridViewComboBoxColumn, value);
  FromPrompt = true;
  Active = false;
  KeepOpen = false;
  Cancelled = false;
  if (edit != null)
    EditText = value;
  if (vStatus != null)
    vStatus.Text = "write from prompt to table";
  if (vDGW.IsCurrentCellInEditMode)
    vDGW.EndEdit();
  if (curRow >= 0 && curRow < vDGW.Rows.Count)
    vDGW.Rows[curRow].Cells[curCell].Value = value;
  ForceClose();
}

public static void Attach(Control control)
{
  DetachEditControl();
  edit = control;
  txt = control as TextBox;
  if (edit == null)
    return;
  edit.PreviewKeyDown += Spec_PreKeyDown;
  edit.KeyDown += Spec_KeyDown;
  edit.KeyPress += Spec_KeyPress;
  edit.KeyUp += Spec_KeyUp;
}

public static void DetachEditControl()
{
  if (edit == null)
    return;
  edit.PreviewKeyDown -= Spec_PreKeyDown;
  edit.KeyDown -= Spec_KeyDown;
  edit.KeyPress -= Spec_KeyPress;
  edit.KeyUp -= Spec_KeyUp;
  edit = null;
  txt = null;
}

public static void PlaceCaretAtStart()
{
  if (edit is TextBoxBase text)
    text.Select(0, 0);
  else if (edit is ComboBox combo)
  {
    combo.SelectionStart = 0;
    combo.SelectionLength = 0;
    combo.DroppedDown = false;
  }
}

static void HideComboDropDown()
{
```

```
    ComboBox combo = edit as ComboBox;
    if (combo != null)
      combo.DroppedDown = false;
  }

  static string EditText
  {
    get
    {
      if (edit is TextBoxBase text)
        return text.Text ?? "";
      ComboBox combo = edit as ComboBox;
      return combo != null ? (combo.Text ?? "") : "";
    }
    set
    {
      if (edit is TextBoxBase text)
        text.Text = value ?? "";
      else if (edit is ComboBox combo)
        combo.Text = value ?? "";
    }
  }

  static int SelectionLength
  {
    get
    {
      if (edit is TextBoxBase text)
        return text.SelectionLength;
      ComboBox combo = edit as ComboBox;
      return combo != null ? combo.SelectionLength : 0;
    }
  }

  static string CellText(int row)
  {
    if (vPrompt == null || row < 0 || row >= vPrompt.Rows.Count)
      return "";
    return (Convert.ToString(vPrompt.Rows[row].Cells[0].Value) ?? "").Trim();
  }

  static void ClearPromptSelection()
  {
    if (vPrompt == null)
      return;
    vPrompt.ClearSelection();
    try
    {
      vPrompt.CurrentCell = null;
    }
    catch (InvalidOperationException)
    {
    }
  }
 }
}
```

2.3. Разметка окна

```
using System.Drawing;
using System.Windows.Forms;

namespace TipTop
{
 partial class Form1
 {
  private System.ComponentModel.IContainer components = null;

  protected override void Dispose(bool disposing)
  {
    if (disposing && (components != null))
      components.Dispose();
    base.Dispose(disposing);
  }

  private void InitializeComponent()
  {
    components = new System.ComponentModel.Container();
    menuStrip1 = new MenuStrip();
    создатьСписокToolStripMenuItem = new ToolStripMenuItem();
    подключитьСписокToolStripMenuItem = new ToolStripMenuItem();
    упорядочитьToolStripMenuItem = new ToolStripMenuItem();
```

```
statusStrip1 = new StatusStrip();
toolStripStatusLabel2 = new ToolStripStatusLabel();
dataGridView2 = new DataGridView();
dgwNamesPrompt = new DataGridView();
menuStrip1.SuspendLayout();
statusStrip1.SuspendLayout();
((System.ComponentModel.ISupportInitialize)dataGridView2).BeginInit();
((System.ComponentModel.ISupportInitialize)dgwNamesPrompt).BeginInit();
SuspendLayout();

menuStrip1.Items.AddRange(new ToolStripItem[]
{
  создатьСписокToolStripMenuItem,
  подключитьСписокToolStripMenuItem,
  упорядочитьToolStripMenuItem
});
menuStrip1.Location = new Point(0, 0);
menuStrip1.Name = "menuStrip1";
menuStrip1.Size = new Size(980, 24);
menuStrip1.TabIndex = 0;
menuStrip1.Text = "menuStrip1";

создатьСписокToolStripMenuItem.Name = "создатьСписокToolStripMenuItem";
создатьСписокToolStripMenuItem.Size = new Size(109, 20);
создатьСписокToolStripMenuItem.Text = "Создать список";
создатьСписокToolStripMenuItem.Click += создатьСписокToolStripMenuItem_Click;

подключитьСписокToolStripMenuItem.Name = "подключитьСписокToolStripMenuItem";
подключитьСписокToolStripMenuItem.Size = new Size(129, 20);
подключитьСписокToolStripMenuItem.Text = "Подключить список";
подключитьСписокToolStripMenuItem.Click += подключитьСписокToolStripMenuItem_Click;

упорядочитьToolStripMenuItem.Name = "упорядочитьToolStripMenuItem";
упорядочитьToolStripMenuItem.Size = new Size(97, 20);
упорядочитьToolStripMenuItem.Text = "Упорядочить";
упорядочитьToolStripMenuItem.Click += упорядочитьToolStripMenuItem_Click;

statusStrip1.Items.AddRange(new ToolStripItem[] { toolStripStatusLabel2 });
statusStrip1.Location = new Point(0, 528);
statusStrip1.Name = "statusStrip1";
statusStrip1.Size = new Size(980, 22);
statusStrip1.TabIndex = 1;
statusStrip1.Text = "statusStrip1";

toolStripStatusLabel2.Name = "toolStripStatusLabel2";
toolStripStatusLabel2.Spring = true;
toolStripStatusLabel2.TextAlign = ContentAlignment.MiddleLeft;
toolStripStatusLabel2.Text = "Готово";

dataGridView2.Anchor = AnchorStyles.Top | AnchorStyles.Bottom | AnchorStyles.Left | AnchorStyles.Right;
dataGridView2.ColumnHeadersHeightSizeMode = DataGridViewColumnHeadersHeightSizeMode.AutoSize;
dataGridView2.Location = new Point(12, 36);
dataGridView2.Name = "dataGridView2";
dataGridView2.RowHeadersWidth = 51;
dataGridView2.Size = new Size(956, 480);
dataGridView2.TabIndex = 2;
dataGridView2.CellBeginEdit += dataGridView2_CellBeginEdit;
dataGridView2.CellEndEdit += dataGridView2_CellEndEdit;
dataGridView2.CellMouseDown += dataGridView2_CellMouseDown;
dataGridView2.CellValidating += dataGridView2_CellValidating;
dataGridView2.DataError += dataGridView2_DataError;
dataGridView2.EditingControlShowing += dataGridView2_EditingControlShowing;
dataGridView2.Scroll += dataGridView2_Scroll;

dgwNamesPrompt.ColumnHeadersVisible = false;
dgwNamesPrompt.RowHeadersVisible = false;
dgwNamesPrompt.Visible = false;
dgwNamesPrompt.AllowUserToAddRows = false;
dgwNamesPrompt.AllowUserToDeleteRows = false;
dgwNamesPrompt.AllowUserToResizeColumns = false;
dgwNamesPrompt.AllowUserToResizeRows = false;
dgwNamesPrompt.BackgroundColor = Color.White;
dgwNamesPrompt.BorderStyle = BorderStyle.FixedSingle;
dgwNamesPrompt.CausesValidation = false;
dgwNamesPrompt.EditMode = DataGridViewEditMode.EditProgrammatically;
dgwNamesPrompt.Location = new Point(40, 80);
dgwNamesPrompt.MultiSelect = false;
dgwNamesPrompt.Name = "dgwNamesPrompt";
dgwNamesPrompt.ReadOnly = true;
dgwNamesPrompt.RowTemplate.Height = 24;
dgwNamesPrompt.ScrollBars = ScrollBars.Vertical;
dgwNamesPrompt.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
dgwNamesPrompt.Size = new Size(180, 28);
dgwNamesPrompt.TabIndex = 3;
dgwNamesPrompt.TabStop = false;

AutoScaleMode = AutoScaleMode.Font;
```

```
    ClientSize = new Size(980, 550);
    Controls.Add(dgwNamesPrompt);
    Controls.Add(dataGridView2);
    Controls.Add(statusStrip1);
    Controls.Add(menuStrip1);
    MainMenuStrip = menuStrip1;
    MinimumSize = new Size(720, 420);
    Name = "Form1";
    StartPosition = FormStartPosition.CenterScreen;
    Text = "ЛР4 — подсказка в экранной таблице";
    menuStrip1.ResumeLayout(false);
    menuStrip1.PerformLayout();
    statusStrip1.ResumeLayout(false);
    statusStrip1.PerformLayout();
    ((System.ComponentModel.ISupportInitialize)dataGridView2).EndInit();
    ((System.ComponentModel.ISupportInitialize)dgwNamesPrompt).EndInit();
    ResumeLayout(false);
    PerformLayout();
   }

   private MenuStrip menuStrip1;
   private ToolStripMenuItem создатьСписокToolStripMenuItem;
   private ToolStripMenuItem подключитьСписокToolStripMenuItem;
   private ToolStripMenuItem упорядочитьToolStripMenuItem;
   private StatusStrip statusStrip1;
   private ToolStripStatusLabel toolStripStatusLabel2;
   private DataGridView dataGridView2;
   private DataGridView dgwNamesPrompt;
  }
}
```

2.4. Съёмка сценариев

```
using System;
using System.Drawing;
using System.Drawing.Imaging;
using System.IO;
using System.Reflection;
using System.Runtime.InteropServices;
using System.Text;
using System.Windows.Forms;

namespace TipTop
{
 static class DemoShots
 {
  const uint PW_RENDERFULLCONTENT = 2;

  [DllImport("user32.dll")]
  static extern bool PrintWindow(IntPtr hwnd, IntPtr hdc, uint flags);

  public static int Run(string dir)
  {
   Directory.CreateDirectory(dir);
   StringBuilder log = new StringBuilder();
   Form1 form = new Form1();
   form.StartPosition = FormStartPosition.Manual;
   form.Location = new Point(40, 30);
   form.TopMost = true;
   form.Show();
   Application.DoEvents();
   try
   {
    DataGridView grid = (DataGridView)Field(form, "dataGridView2");
    DataGridView prompt = (DataGridView)Field(form, "dgwNamesPrompt");
    ToolStripStatusLabel status = (ToolStripStatusLabel)Field(form, "toolStripStatusLabel2");

    Shot(form, dir, "01-start.png");
    Note(log, "01", status, grid, prompt);

    Click(form, "создатьСписокToolStripMenuItem");
    Click(form, "подключитьСписокToolStripMenuItem");
    Click(form, "упорядочитьToolStripMenuItem");
    Application.DoEvents();
    Shot(form, dir, "02-menu-sorted.png");
    Note(log, "02", status, grid, prompt);

    Edit(grid, 0, 1);
    Shot(form, dir, "03-prompt-sorted.png");
    Note(log, "03", status, grid, prompt);

    TextBox editor = grid.EditingControl as TextBox;
```

```
  Prompt.Spec_KeyPress(editor, new KeyPressEventArgs('C'));
  editor.Text = "C";
  editor.SelectionStart = editor.Text.Length;
  Application.DoEvents();
  Shot(form, dir, "04-type-c.png");
  Note(log, "04", status, grid, prompt);

  Prompt.Spec_KeyDown(editor, new KeyEventArgs(Keys.Down));
  Application.DoEvents();
  Shot(form, dir, "05-arrow-down.png");
  Note(log, "05", status, grid, prompt);

  Prompt.Spec_KeyDown(editor, new KeyEventArgs(Keys.Enter));
  Application.DoEvents();
  Shot(form, dir, "06-enter-hint.png");
  Note(log, "06", status, grid, prompt);

  Edit(grid, 1, 1);
  editor = grid.EditingControl as TextBox;
  editor.Text = "Neon";
  editor.SelectionStart = editor.Text.Length;
  Prompt.Spec_KeyDown(editor, new KeyEventArgs(Keys.Enter));
  Application.DoEvents();
  Shot(form, dir, "07-store-new.png");
  Note(log, "07", status, grid, prompt);

  editor = grid.EditingControl as TextBox;
  Prompt.Spec_KeyDown(editor, new KeyEventArgs(Keys.Escape));
  Application.DoEvents();
  Shot(form, dir, "08-escape.png");
  Note(log, "08", status, grid, prompt);

  Edit(grid, 2, 1);
  editor = grid.EditingControl as TextBox;
  editor.Text = "";
  Prompt.Spec_KeyDown(editor, new KeyEventArgs(Keys.Enter));
  Application.DoEvents();
  Shot(form, dir, "09-empty-enter.png");
  Note(log, "09", status, grid, prompt);

  Edit(grid, 3, 1);
  if (prompt.Rows.Count > 0)
    prompt.FirstDisplayedScrollingRowIndex = 0;
  Application.DoEvents();
  Rectangle row = prompt.GetRowDisplayRectangle(0, true);
  if (row.Width <= 0 || row.Height <= 0)
    row = new Rectangle(4, 4, 40, 20);
  MethodInfo mouseDown = typeof(Prompt).GetMethod("prompt_MouseDown", BindingFlags.NonPublic | BindingFlags.Static);
  mouseDown.Invoke(null, new object[]
  {
    prompt,
    new MouseEventArgs(MouseButtons.Left, 1, Math.Max(2, row.Left + 6), Math.Max(2, row.Top + 4), 0)
  });
  Application.DoEvents();
  Shot(form, dir, "10-mouse.png");
  Note(log, "10", status, grid, prompt);

  Edit(grid, 0, 2);
  Application.DoEvents();
  Shot(form, dir, "11-type-column.png");
  Note(log, "11", status, grid, prompt);

  if (grid.EditingControl is ComboBox combo)
  {
    combo.Text = "Насекомое";
    combo.SelectionStart = combo.Text.Length;
    combo.DroppedDown = false;
    Prompt.Spec_KeyDown(combo, new KeyEventArgs(Keys.Enter));
    Application.DoEvents();
    if (!combo.IsDisposed)
      combo.DroppedDown = false;
  }
  Application.DoEvents();
  Shot(form, dir, "12-new-type.png");
  Note(log, "12", status, grid, prompt);

  Edit(grid, 0, 0);
  Application.DoEvents();
  Shot(form, dir, "13-nn-column.png");
  Note(log, "13", status, grid, prompt);
}
finally
{
  form.Close();
}
File.WriteAllText(Path.Combine(dir, "demo.log"), log.ToString(), Encoding.UTF8);
Console.WriteLine(log.ToString());
```

```
    return 0;
  }

  static void Edit(DataGridView grid, int row, int column)
  {
   if (grid.IsCurrentCellInEditMode)
     grid.EndEdit();
   grid.CurrentCell = grid.Rows[row].Cells[column];
   Application.DoEvents();
   grid.BeginEdit(false);
   Application.DoEvents();
  }

  static void Note(StringBuilder log, string id, ToolStripStatusLabel status, DataGridView grid, DataGridView prompt)
  {
   string cell = "";
   if (grid.CurrentCell != null)
     cell = Convert.ToString(grid.CurrentCell.Value);
   string selected = "(none)";
   if (prompt.SelectedRows.Count > 0)
     selected = Convert.ToString(prompt.SelectedRows[0].Cells[0].Value);
   log.AppendLine(id
     + " status=[" + status.Text + "]"
     + " editing=" + grid.IsCurrentCellInEditMode
     + " cell=[" + cell + "]"
     + " promptVisible=" + prompt.Visible
     + " promptRows=" + prompt.Rows.Count
     + " selected=[" + selected + "]"
     + " names=[" + string.Join(",", Form1.vns.ConvertAll(n => n.Name)) + "]");
  }

  static void Shot(Form form, string dir, string name)
  {
   form.BringToFront();
   form.Activate();
   form.Refresh();
   Application.DoEvents();
   DataGridView prompt = null;
   foreach (Control control in form.Controls)
   {
    DataGridView grid = control as DataGridView;
    if (grid != null && grid.Name == "dgwNamesPrompt")
      prompt = grid;
   }
   if (prompt != null && prompt.Visible && prompt.Rows.Count > 0 && prompt.Rows.Count <= 8)
   {
    try { prompt.FirstDisplayedScrollingRowIndex = 0; }
    catch (InvalidOperationException) { }
   }
   Rectangle bounds = form.Bounds;
   using (Bitmap bmp = new Bitmap(bounds.Width, bounds.Height))
   {
    using (Graphics g = Graphics.FromImage(bmp))
    {
     IntPtr hdc = g.GetHdc();
     PrintWindow(form.Handle, hdc, PW_RENDERFULLCONTENT);
     g.ReleaseHdc(hdc);
    }
    bmp.Save(Path.Combine(dir, name), ImageFormat.Png);
   }
  }

  static void Click(Form form, string field)
  {
   ToolStripMenuItem item = (ToolStripMenuItem)Field(form, field);
   item.PerformClick();
  }

  static object Field(Form form, string name)
  {
   return form.GetType().GetField(name, BindingFlags.Instance | BindingFlags.NonPublic | BindingFlags.Public).GetValue(form);
  }
 }
}
```

3. Описание написанного кода

3.1. Form1

Конструктор собирает три столбца и восемь пустых строк. Режим редактирования — EditOnKeystrokeOrF2: первый щелчок выделяет ячейку, ввод с клавиатуры или F2 открывает редактор. Повторный щелчок по уже текущей ячейке вызывает BeginEdit(false) и ставит каретку в начало строки. Это сценарий щелчка левой кнопкой при уже выделенной ячейке. «Создать список» добавляет Argon, Xenon и Cripton, не дублируя уже имеющиеся имена. «Подключить список» снимает прежнюю привязку и назначает vbs источником dgwNamesPrompt; Rows.Clear() вызывается только пока источника ещё нет. «Упорядочить» строит новый список по возрастанию Name, записывает его и в vns, и в vbs. Если в этот момент открыта подсказка столбца «Название», окно подсказки пересчитывает размер. GetList, SetList и GetSource отделяют столбец 1 от остальных. CellBeginEdit открывает подсказку для любого столбца. CellEndEdit переносит значение в список, если редактирование не отменено и значение пришло не из щелчка по подсказке. Флаг KeepOpen оставляет окно видимым после пустого Enter. Смена текущей ячейки закрывает подсказку, когда редактирование уже закончено и текущая ячейка — другая. EditingControlShowing подключает общие обработчики и к текстовому редактору, и к ComboBox. У комбинированного редактора стиль DropDown, чтобы можно было набрать новое слово, а штатный выпадающий список в этот момент закрывается: на экране остаётся окно подсказки. Стрелка комбинированного столбца по-прежнему открывает штатный список.

3.2. Names и Prompt

Names — одна строка подсказки, свойство Name. Таблица подсказки показывает его автоматически через привязку. OpenPrompt запоминает ячейку, привязывает список этого столбца и включает обработчики один раз. Повторный вход в ту же или другую ячейку заново ставит окно и не накапливает подписки: Wire проверяет флаг wired. Для пустого комбинированного столбца список заполняется его Items. AdjustPrompt переводит прямоугольник ячейки в координаты формы, ограничивает окно областью между меню и строкой состояния

и при нехватке места снизу переносит его вверх. Ширина колонки подсказки — Fill, поэтому строка занимает всё окно. Spec_KeyPress строит текст так, как он будет выглядеть после текущей литеры, и вызывает Advance. Если в редакторе есть выделение, текст сначала очищается: первая литера при входе в ячейку заменяет выделенное, как в образце. Advance идёт с нулевой строки, пока введённый текст больше значения строки, и выделяет первую строку, которая не меньше введённого. Пустой текст выделение снимает. Spec_KeyUp для Backspace и Delete пересчитывает выделение по уже изменённому тексту. CommitTypedText различает пустой Enter и Enter с текстом. Пустой ввод ставит KeepOpen и завершает редактирование. Непустой вызывает StoreText, очищает выделение подсказки и оставляет редактор открытым. StoreText отбрасывает пустую строку и точное повторение, сортирует список, меняет источник привязки и для комбинированного столбца добавляет пункт в Items. ApplyToCell записывает выбранную строку в редактор, завершает редактирование и прячет подсказку. Из MouseDown вызов откладывается через BeginInvoke, чтобы не завершать редактирование внутри сообщения мыши. CancelEditAndHide возвращает в редактор прежнее значение ячейки, отменяет ввод и закрывает окно. Если отмена из обработчика клавиши оказывается вложенной, завершение переносится в очередь сообщений формы.

3.3. DemoShots

Класс прогоняет окно по тем же обработчикам, которыми пользуется оператор: команды меню, BeginEdit, Spec_KeyPress, Spec_KeyDown и prompt_MouseDown. После каждого контрольного состояния PrintWindow сохраняет снимок окна. Каталог задаётся аргументом --demo-out.

4. Описание тестовых сценариев

Окно проверяется одним прогоном:

```
dotnet run --project TipTop -- --demo-out=..\report\images
```

Прогон выполняется из каталога TipTop. Снимки лежат в report/images и приведены ниже. В строке состояния остаётся текст, по которому видно, какое действие только что выполнено. Сценарии идут одной цепочкой.

1. старт: три столбца, пустые строки, подсказка скрыта;
2. меню: создать список, подключить его и упорядочить;
3. открытие подсказки под ячейкой «Название»: Argon, Cripton, Xenon;
4. литера C сдвигает выделение на Cripton;
5. стрелка вниз сдвигает выделение на Xenon, Enter переносит Xenon в ячейку и прячет подсказку;
6. в соседней ячейке набирается Neon и подтверждается Enter: строка попадает в список по алфавиту, редактирование продолжается, подсказка открыта;
7. Escape отменяет набор и прячет подсказку, уже записанный Xenon остаётся в таблице;
8. пустой Enter заканчивает редактирование и оставляет подсказку видимой;
9. щелчок по первой строке подсказки переносит Argon в текущую ячейку;
10. столбец «Тип» открывает собственный список из четырёх штатных значений;
11. слово «Насекомое» добавляется в этот список, сортируется и остаётся допустимым значением комбинированного столбца;
12. столбец NN открывает свой пустой список и не показывает названия.

5. Тестовые сценарии

Ниже — состояния окна после прогона DemoShots. Ожидаемый состав списка и текст ячейки совпали с фактическими.

5.1. Старт

Шаги: запустить приложение.

Ожидание: меню из трёх команд, столбцы NN, «Название» и «Тип», восемь пустых строк и строка для новой записи, подсказка скрыта, в статусе памятка по клавишам.

Факт: совпало, рисунок 1.

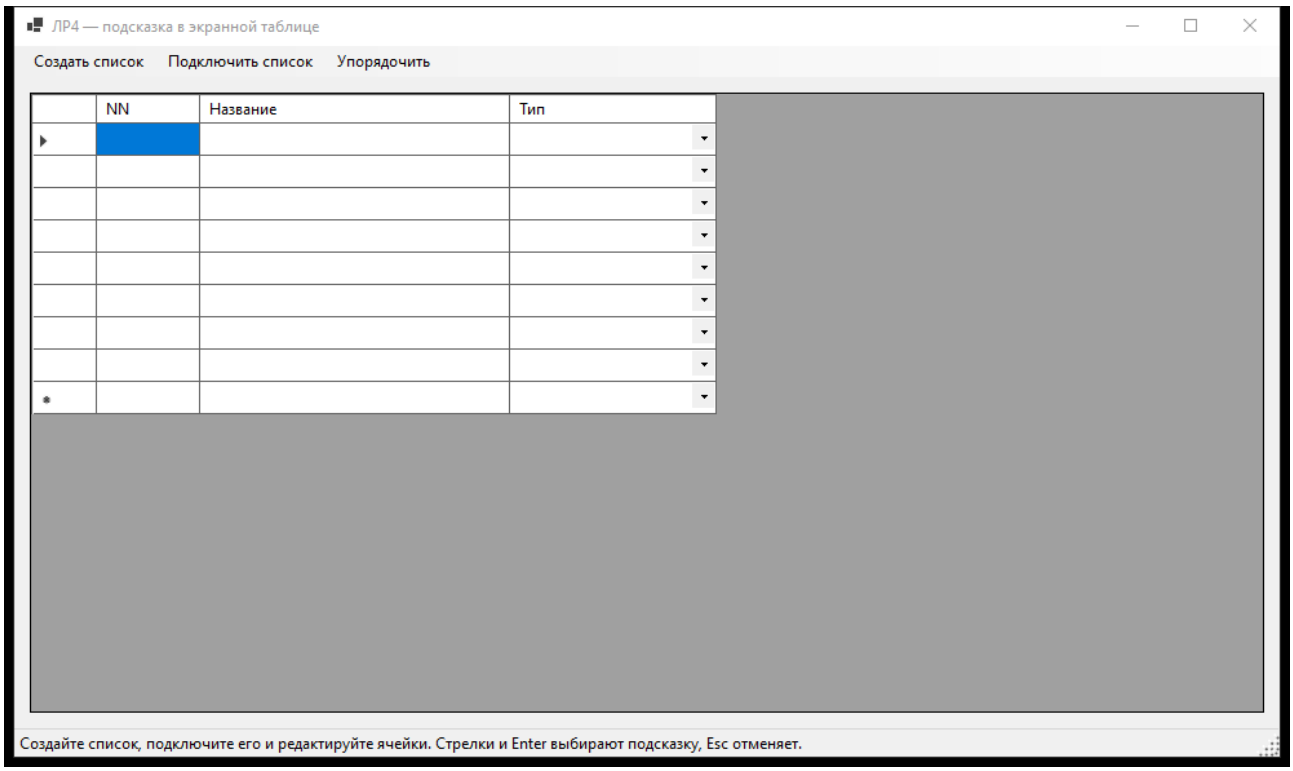


Рис. 1: Окно при запуске

5.2. Команды меню

Шаги: «Создать список», «Подключить список», «Упорядочить». Ожидание: статус «Список упорядочен». Подсказка ещё скрыта: команды только готовят список столбца «Название». Порядок имён — Argon, Cripton, Xenon. Факт: совпало, рисунок 2.

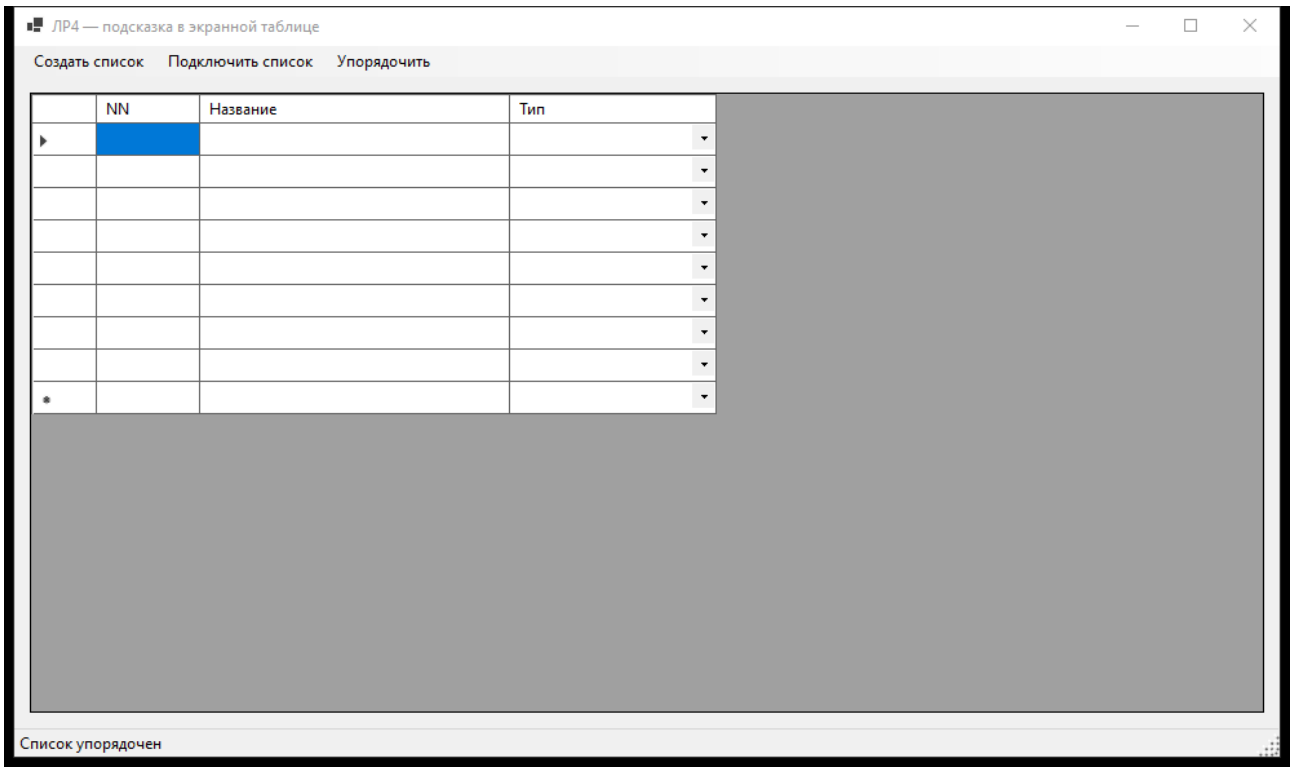


Рис. 2: Список создан, подключён и упорядочен

5.3. Подсказка под ячейкой «Название»

Шаги: начать редактирование первой ячейки столбца «Название». Ожидание: под ячейкой окно без заголовков, строки Argon, Cripton, Xenon, ячейка в режиме редактирования, статус сообщает, что редактор таблицы создан. Факт: совпало, рисунок 3.

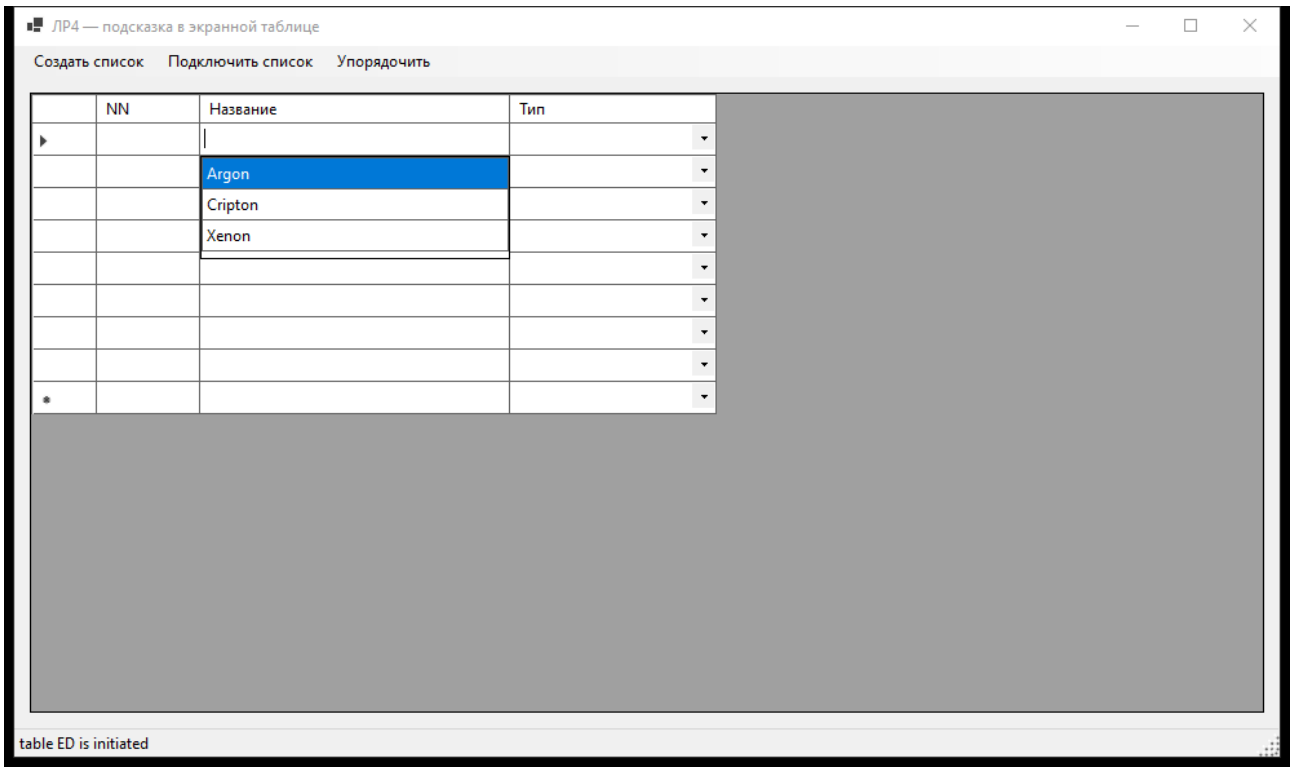


Рис. 3: Упорядоченная подсказка под столбцом «Название»

5.4. Продвижение селекции при вводе

Шаги: нажать C.

Ожидание: в ячейке литера C, выделена первая строка, которая не меньше этого текста, — Cripton. Argon остаётся выше, Xenon ниже.

Факт: совпало, рисунок 4.

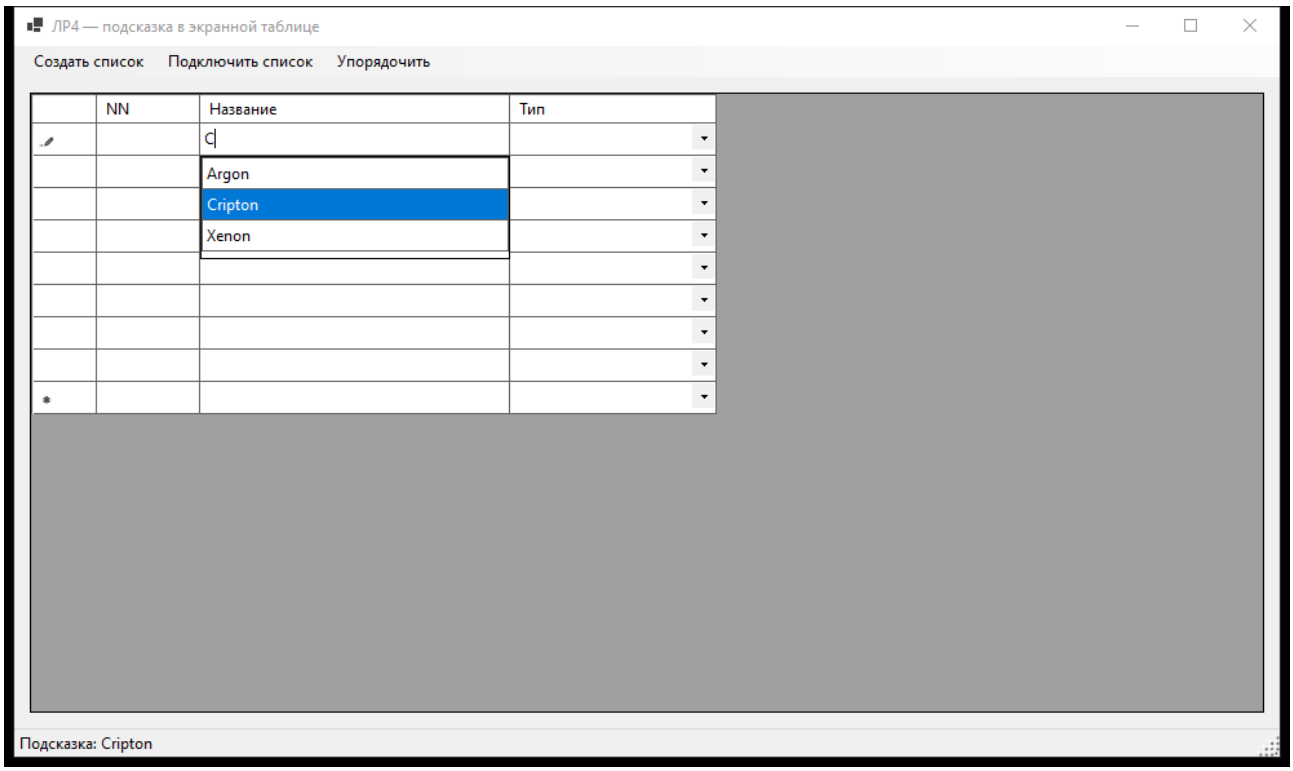


Рис. 4: После литеры C выделена строка Cripton

5.5. Стрелка вниз

Шаги: нажать стрелку вниз.

Ожидание: выделение переходит на Xenon, набранная литера C в ячейке сохраняется, статус показывает «Подсказка: Xenon».

Факт: совпало, рисунок 5.

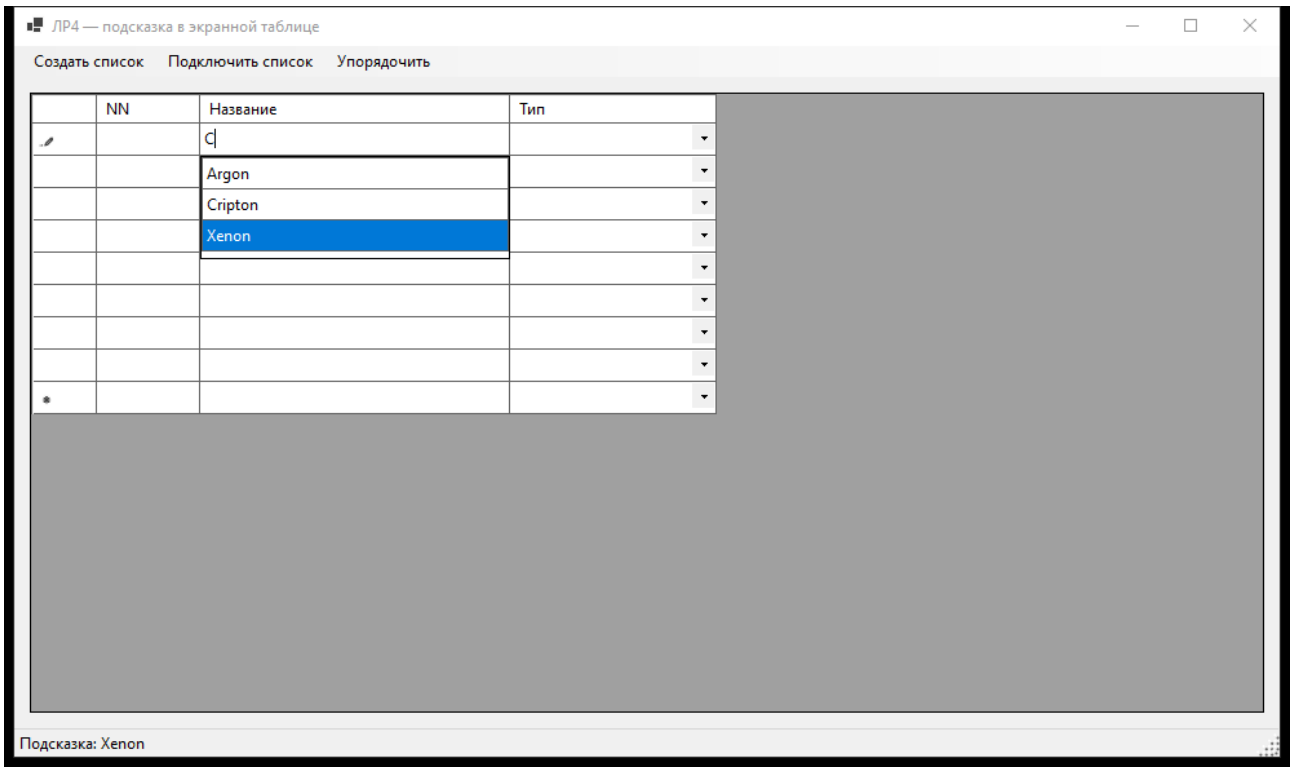


Рис. 5: Стрелка вниз переводит выделение на Xenon

5.6. Enter переносит строку подсказки

Шаги: нажать Enter после стрелки.

Ожидание: в ячейке Xenon, редактирование закончено, подсказка скрыта, статус — write from prompt to table.

Факт: совпало, рисунок 6.

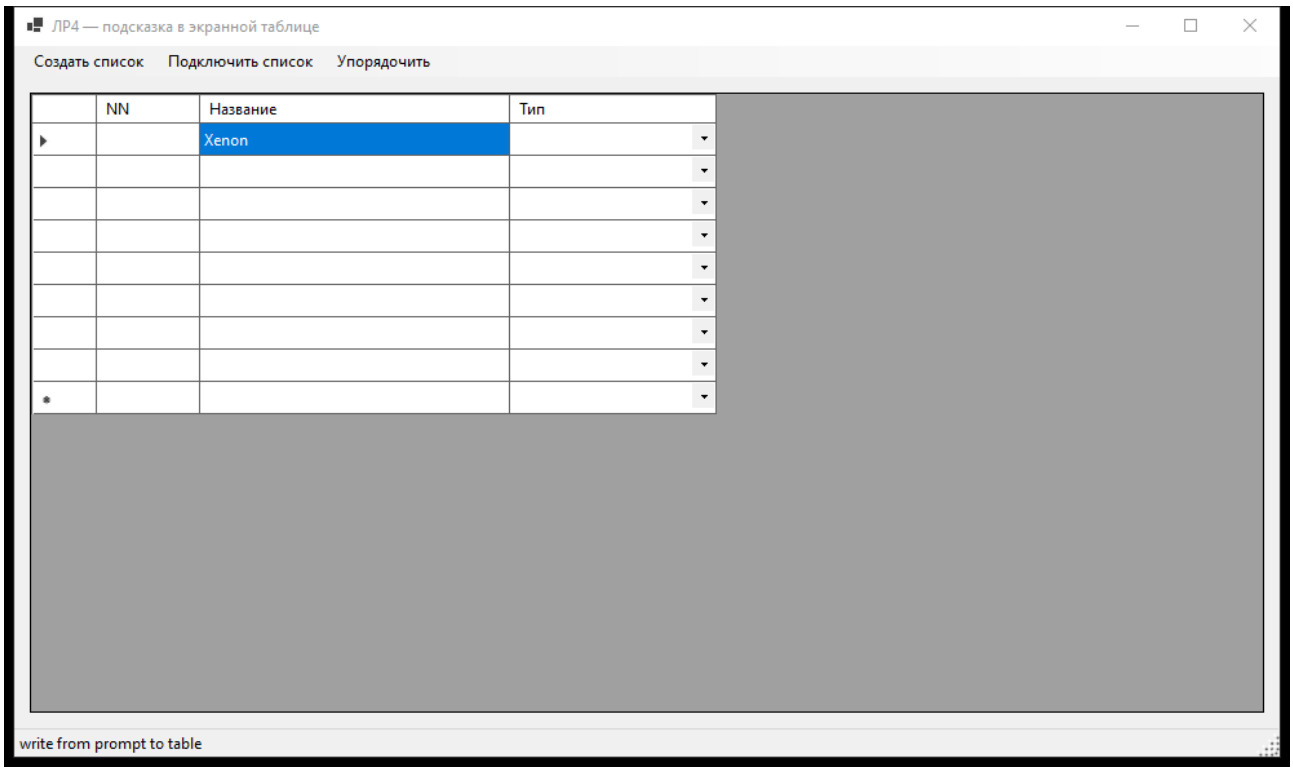


Рис. 6: Xenon перенесён из подсказки в ячейку

5.7. Новая строка попадает в подсказку

Шаги: открыть следующую ячейку «Название», набрать Neon, нажать Enter, не двигая выделение стрелками.

Ожидание: Neon остаётся в редакторе, подсказка открыта и упорядочена: Argon, Cripton, Neon, Xenon. Статус — «Строка занесена в подсказку». Повтора Neon нет.

Факт: совпало, рисунок 7.

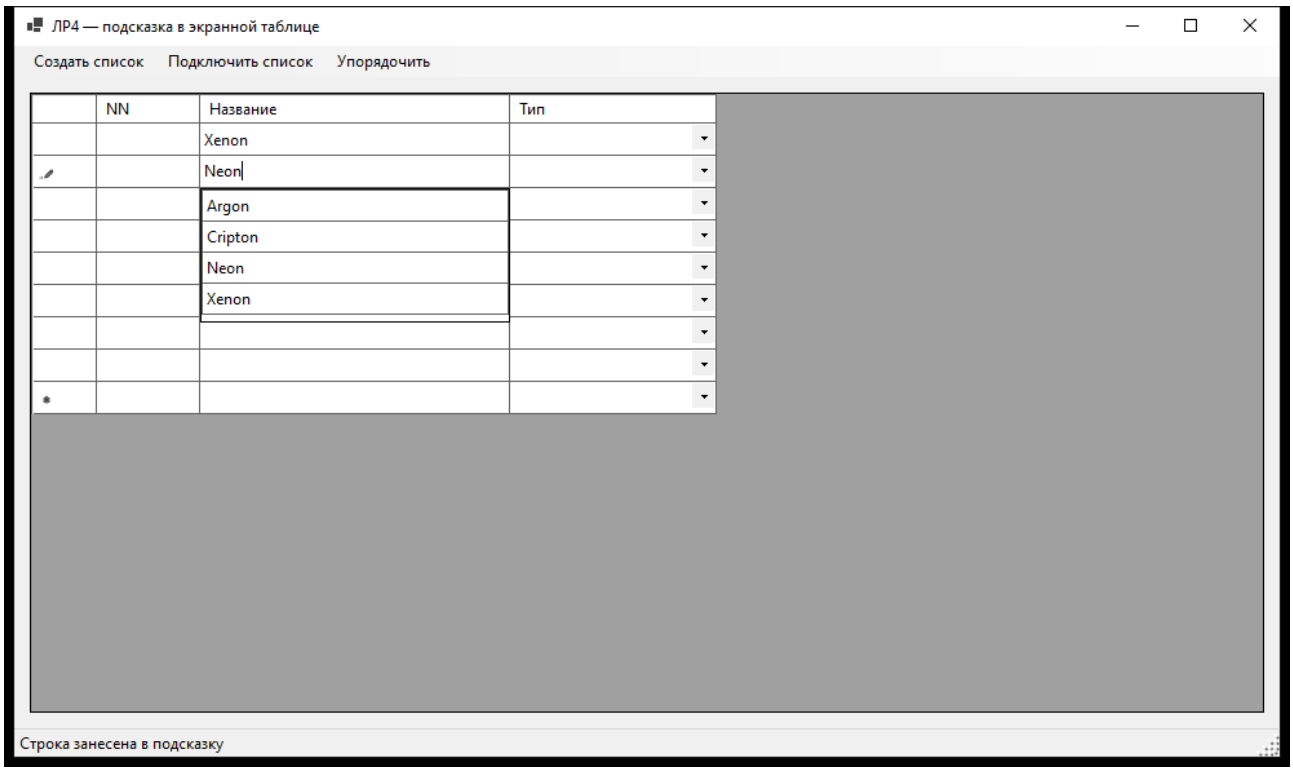


Рис. 7: Neon добавлен в список и стоит между Cripton и Xenon

5.8. Отказ по Escape

Шаги: при открытой подсказке нажать Escape.

Ожидание: редактирование прекращено, набранный текст в ячейку не записывается, подсказка скрыта. Xenon в первой строке на месте. Статус — «Подсказка скрыта».

Факт: совпало, рисунок 8.

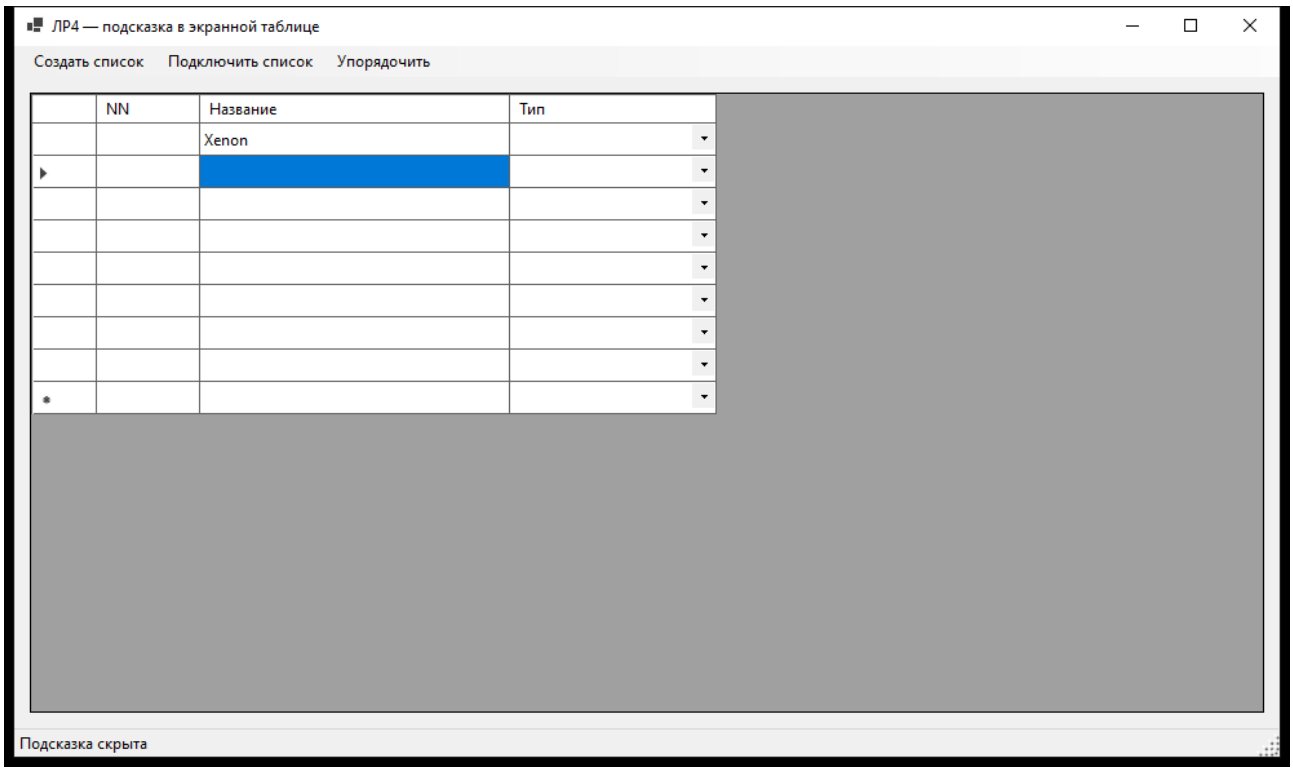


Рис. 8: Escape скрыл подсказку и не записал черновик

5.9. Пустой Enter оставляет подсказку

Шаги: открыть пустую ячейку «Название» и нажать Enter.

Ожидание: редактирование закончено, подсказка остаётся под ячейкой и показывает уже накопленный список, включая Neon. Статус — «Ячейка закрыта, подсказка видима».

Факт: совпало, рисунок 9.

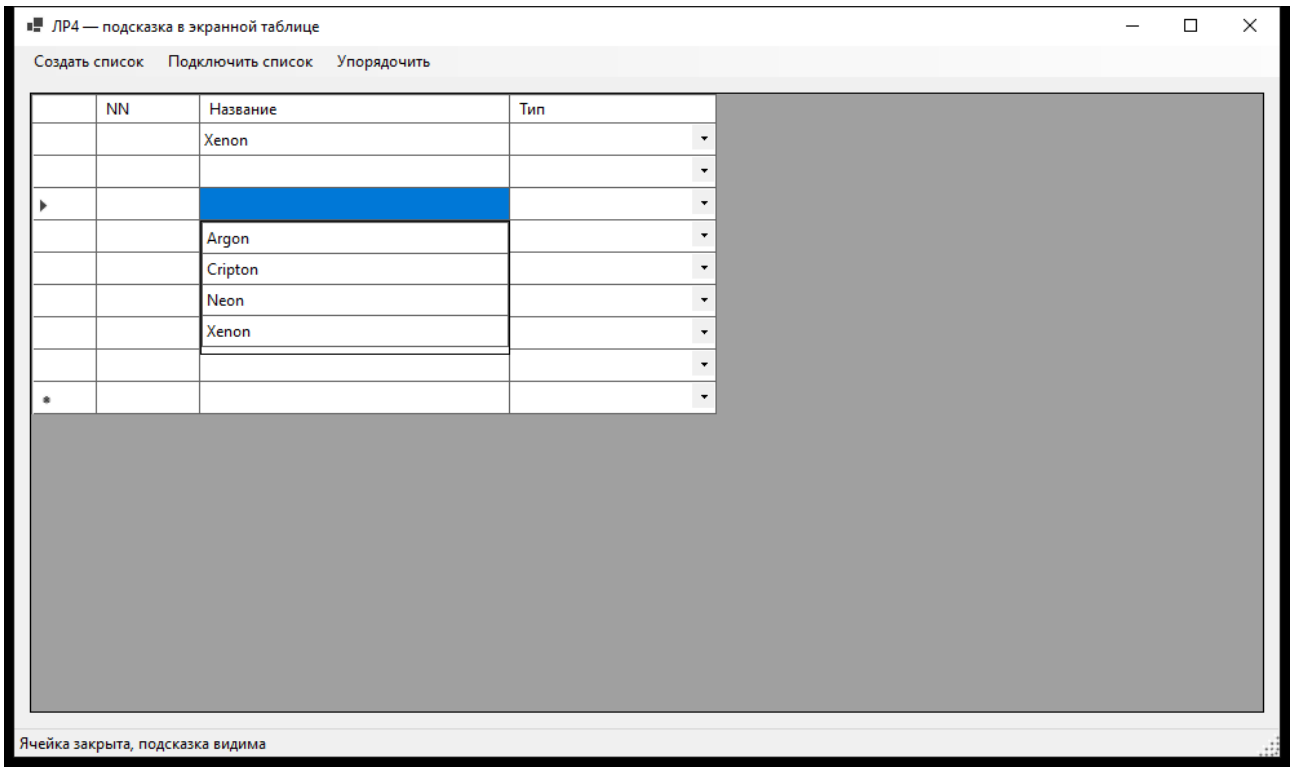


Рис. 9: Пустой Enter закрыл ячейку и оставил подсказку

5.10. Щелчок по строке подсказки

Шаги: начать редактирование другой ячейки «Название» и нажать левую кнопку на первой строке подсказки.

Ожидание: в ячейку попадает Argon, подсказка скрывается, статус — write from prompt to table.

Факт: совпало, рисунок 10.

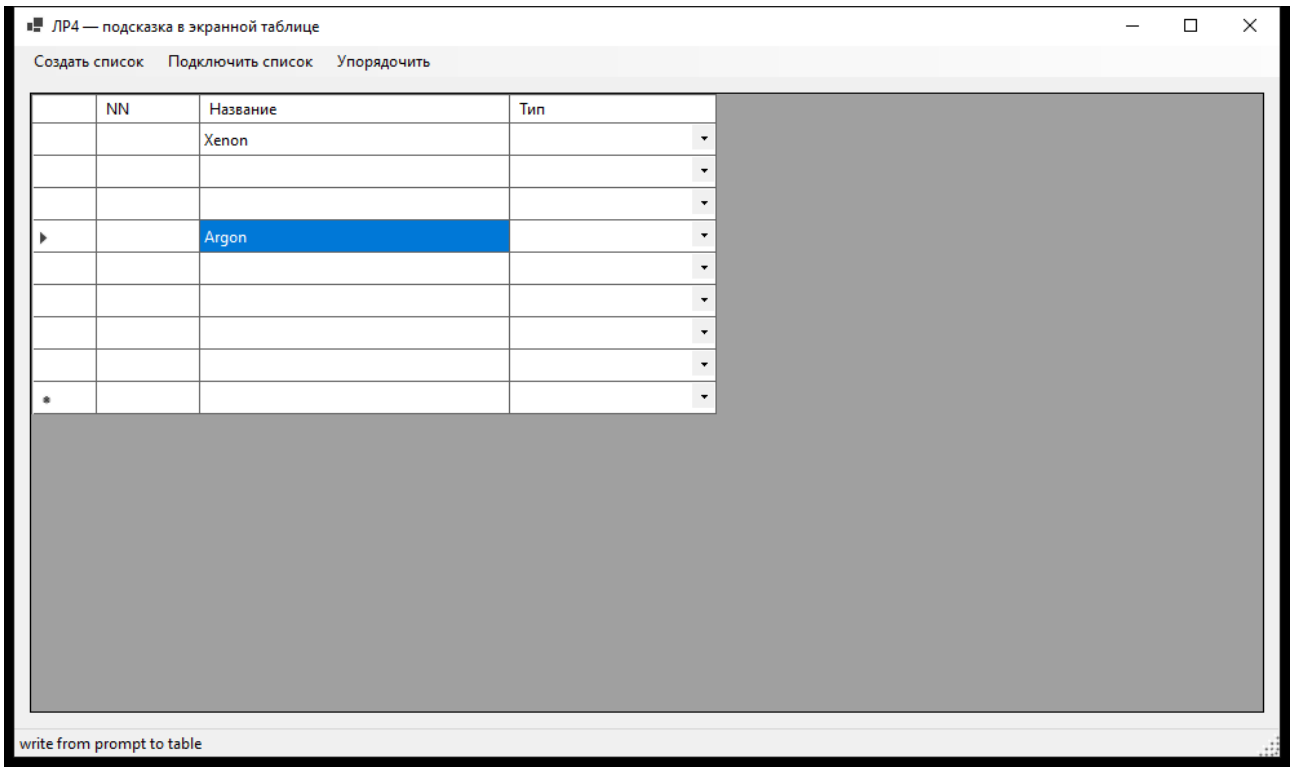


Рис. 10: Щелчок по строке Argon перенёс её в ячейку

5.11. Подсказка столбца «Тип»

Шаги: начать редактирование ячейки «Тип».

Ожидание: под ячейкой собственный список из штатных пунктов комбинированного столбца: Зверь, Птица, Рыба, Гад. Список названий в это окно не попадает. Редактор допускает ввод текста.

Факт: совпало, рисунок 11.

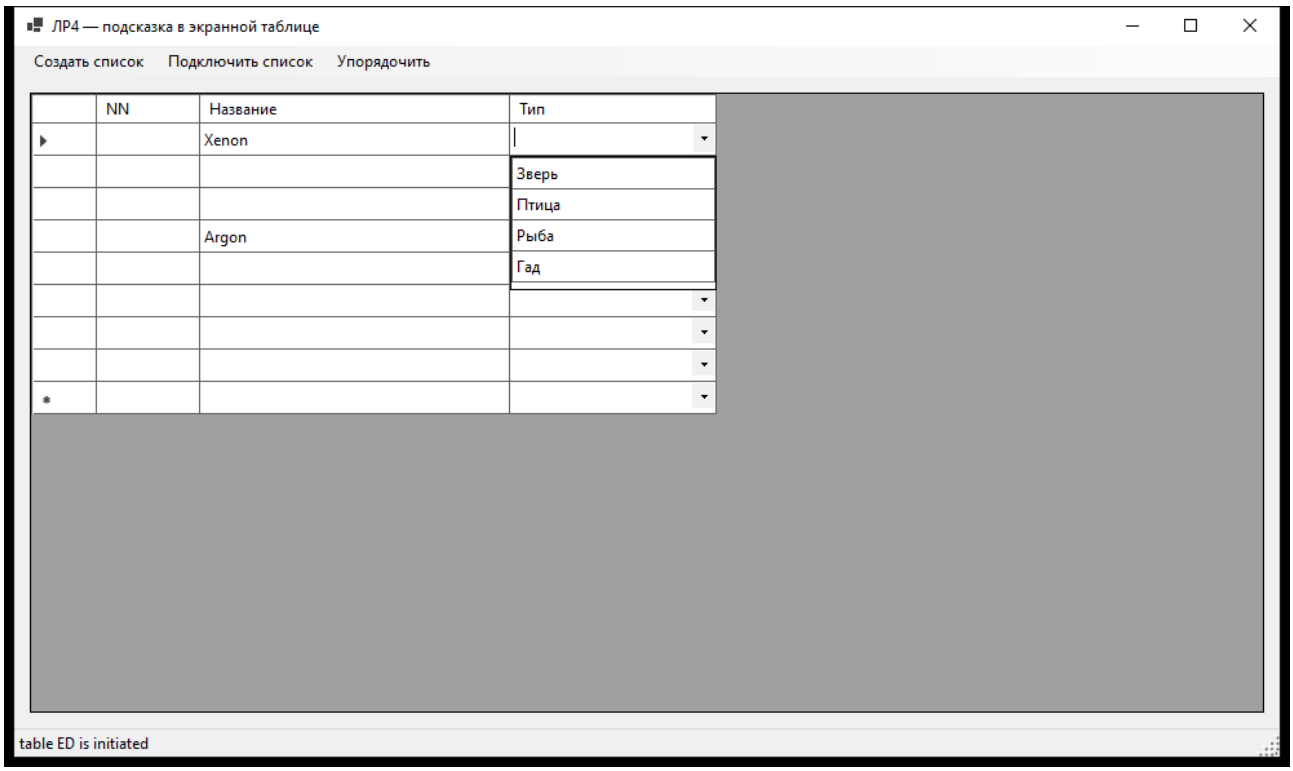


Рис. 11: Столбец «Тип» открывает свой список

5.12. Расширение комбинированного столбца

Шаги: набрать «Насекомое» и нажать Enter.

Ожидание: слово остаётся в ячейке, подсказка столбца «Тип» упорядочена: Гад, Зверь, Насекомое, Птица, Рыба. Значение добавлено и в пункты ComboBoxColumn, поэтому ячейка его принимает. Статус — «Строка занесена в подсказку».

Факт: совпало, рисунок 12.

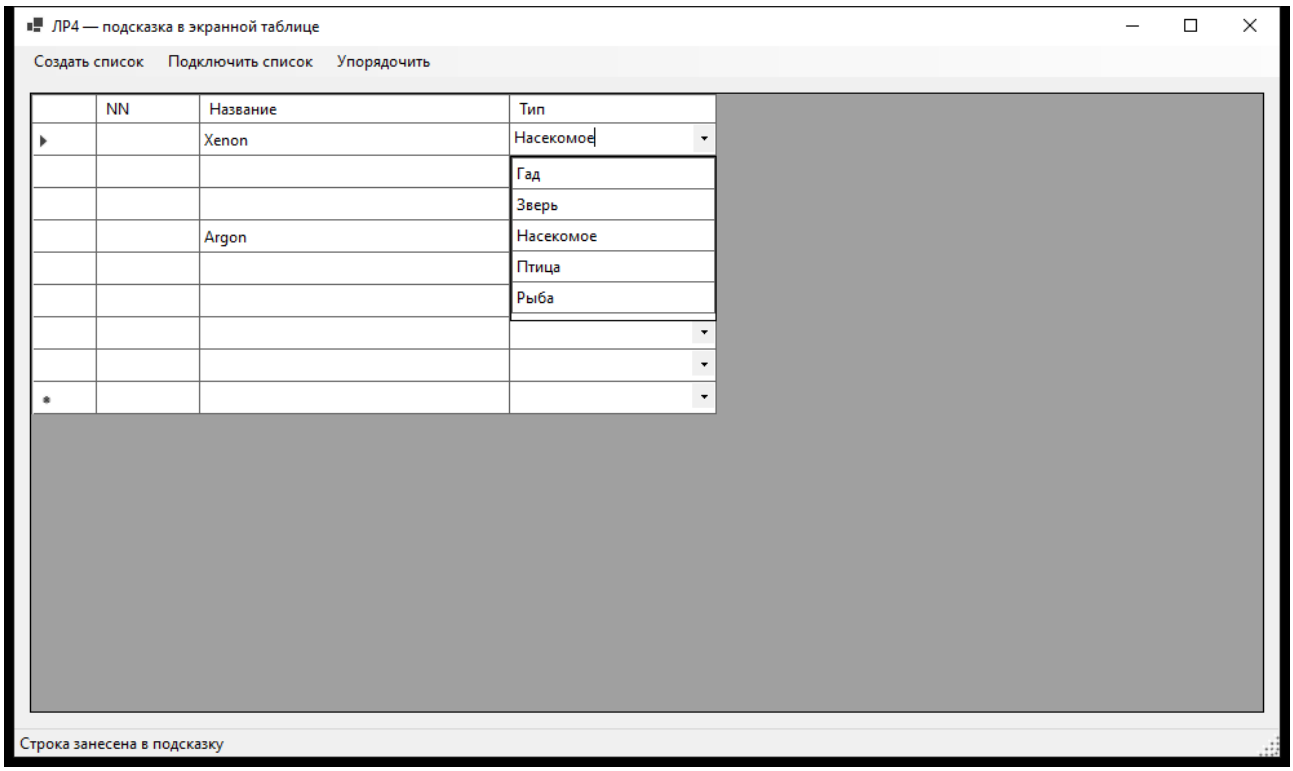


Рис. 12: «Насекомое» добавлено в подсказку столбца «Тип»

5.13. Отдельный список столбца NN

Шаги: начать редактирование ячейки NN.

Ожидание: под ячейкой пустое окно. Названия и типы в него не подставляются: у столбца ещё нет своих строк.

Факт: совпало, рисунок 13.

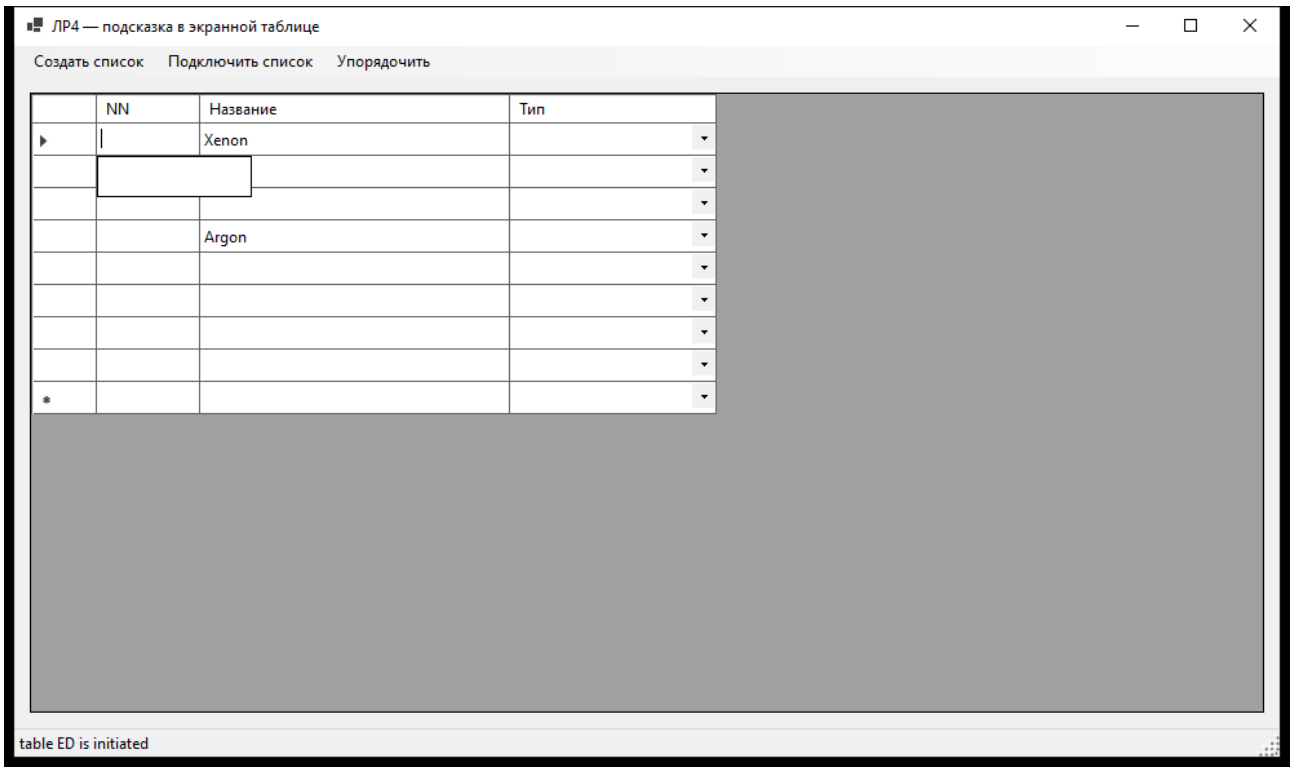


Рис. 13: Столбец NN открывает пустую подсказку

6. Документация

6.1. Руководство оператора

Запуск: dotnet run --project TipTop из каталога лабораторной либо открытие TipTop/TipTop.sln и запуск из Visual Studio. Открывается одно окно. Список для столбца «Название» готовится меню, в таком порядке: «Создать список», «Подключить список», «Упорядочить». Создать список можно и повторно: Argon, Xenon и Cripton второй раз не добавляются. «Упорядочить» сортирует только этот список. Списки NN и «Тип» команда не меняет: они сортируются сами в момент добавления новой строки. Ячейку выделяют щелчком. Редактирование начинается со второго щелчка по ней, с клавиши F2 или с первой буквы. Подсказка появляется под ячейкой. Буквы набираются в ячейку, а выделение в списке идёт к подходящей строке. Стрелки вверх и вниз двигают это выделение, не выходя из ячейки. Enter после стрелки или щелчок по строке подсказки записывает выбранное значение и закрывает подсказку. Enter без стрелок, если в ячейке есть текст, добавляет этот текст в подсказку текущего столбца и оставляет ячейку открытой. Пустой Enter закрывает редактирование и не прячет подсказку. Escape отменяет набранное и прячет подсказку. Переход в другую ячейку

тоже прячет подсказку. В столбце «Тип» одновременно есть штатный список и окно подсказки. Новое слово, подтверждённое Enter, в дальнейшем можно выбрать и там, и там.

6.2. Клавиши

Клавиша	Действие во время редактирования
буквы и другие печатные символы	ввод в ячейку и сдвиг выделения подсказки
стрелка вверх, стрелка вниз	предыдущая или следующая строка подсказки
Enter после стрелки	перенести выделенную строку в ячейку и закрыть подсказку
Enter с набранным текстом	занести текст в подсказку, оставить ячейку в редактировании
Enter в пустой ячейке	закончить редактирование, подсказку не скрывать
Escape	отменить ввод и скрыть подсказку
Backspace, Delete	стереть символ и заново найти строку подсказки
щелчок по строке подсказки	перенести эту строку в ячейку

6.3. Сборка

Нужен SDK .NET 10 с поддержкой Windows Forms. Из каталога TipTop:

```
dotnet build TipTop.csproj
dotnet run
```

Снимки сценариев повторно снимаются так:

```
dotnet run -- --demo-out=..\report\images
```

Отчёт собирается XeLaTeX из каталога report. Нужны пакеты polyglossia, fontspec, fvextra, шрифты Times New Roman и Courier New:

```
xelatex lab4_report.tex
xelatex lab4_report.tex
```

Второй проход нужен для оглавления и ссылок на рисунки.

6.4. Ограничения

Сравнение строк зависит от текущей локали и учитывает регистр: «argon» и «Argon» — разные строки подсказки. Меню «Упорядочить» сортирует только список столбца «Название». Стрелки двигают выделение в окне подсказки и не переводят в него фокус, поэтому набор текста не прерывается. Enter принимает выделенную строку только после стрелки; Enter сразу после набора заносит в список сам набранный текст. Каретка остаётся в конце набора, середина строки отдельно не правится. Штатный список столбца «Тип» при начале редактирования закрывается, чтобы не закрывать окно подсказки; его открывает стрелка комбинированной ячейки. Пустая подсказка — небольшое пустое окно, а не сообщение об ошибке.

6.5. Состав поставки

Путь	Назначение
TipTop/TipTop.sln	решение для Visual Studio
TipTop/TipTop.csproj	проект Windows Forms, .NET 10
TipTop/Program.cs	запуск окна или съёмки сценариев
TipTop/Form1.cs	форма, класс Names, класс Prompt
TipTop/Form1.Designer.cs	меню, таблицы, строка состояния
TipTop/DemoShots.cs	прогон и снимки сценариев
report/lab4_report.tex	настоящий отчёт
report/images	снимки тринадцати сценариев

Заключение

По методичке собрано окно с рабочей таблицей и отдельным окном подсказки. Штатный комбинированный столбец «Тип» сохранён и дополнен тем же механизмом подсказки, что и текстовые столбцы. Список ведётся отдельно для каждого столбца, пополняется новыми строками без повторов и сортируется. Выбор из подсказки выполняется стрелками и Enter либо щелчком, отказ — клавишей Escape. Цепочка из тринадцати состояний окна снята прогоном DemoShots: от пустой таблицы и команд меню до переноса значения из подсказки, добавления новой строки и отдельного списка у каждого столбца.